

Aegis Learn: A Retrieval-Augmented Generative AI Framework for Personalized, Bloom's-Taxonomy-Aligned Learning

Masuma Fathema Avinash Yarukala Madhav Ghattamaneni K V S Jaswanth

Department of CSE — 2420030400, 2420030568, 2420030521, 2420030467

KL University, Hyderabad, India

Under the guidance of Dr. K. Swanthana, Department of Computer Science and Engineering.

Abstract—

Static textbooks give every learner the same page regardless of what they already know, how they learn best, or which cognitive level they are working at. This paper presents Aegis Learn, a retrieval-augmented generative AI framework that turns a fixed textbook corpus into interactive, personalized study support. The system indexes reference material from OpenStax, NCERT, and a Khan Academy algebra corpus, retrieves the passages most relevant to a learner's query using cosine-similarity vector search, and routes the retrieved context together with a cognitive-level tag drawn from Bloom's Taxonomy into a structured prompt for Gemini 1.5 Flash. A React, TypeScript, and Vite front end exposes student, educator, and administrator surfaces, while a performance-tracking module records mastery per Bloom's level and proposes a remedial roadmap. As of this writing the project is in Phase 4 of development, with the interface, prompt templates, and API integration complete and the retrieval analyzer, quiz engine, and mastery dashboard in progress; we report the design, the completed components, and a candid account of what remains before quantitative results can be produced.

Index Terms—personalized learning, retrieval-augmented generation, large language models, Bloom's Taxonomy, vector similarity search, Gemini, intelligent tutoring, educational technology

I. Introduction

Textbooks and static course material are written for an average learner, yet real classrooms contain students who differ in prior knowledge, pace, and the cognitive demand they are ready for. Bloom's Taxonomy [1], later revised by Anderson and Krathwohl [2], organizes learning objectives into six increasingly demanding cognitive levels: Remembering, Understanding, Applying, Analyzing, Evaluating, and Creating. Instructors use this hierarchy routinely, but manually producing level-appropriate notes and quizzes for every topic and learner does not scale.

Large language models can generate fluent explanations on demand, but a model answering purely from parametric memory can drift from the specific textbook a student uses or invent facts. Retrieval-augmented generation (RAG) [3] addresses this by grounding generation in passages retrieved from a trusted corpus at query time.

This paper presents Aegis Learn, a framework that combines textbook-grounded retrieval with Bloom's-Taxonomy-aware prompting to generate personalized study guides, quizzes, and remedial recommendations. The contributions are:

- A retrieval-augmented pipeline that grounds generated study notes and quizzes in a curated textbook corpus.
- A Bloom's-Taxonomy routing layer that maps a learner's request to one of six cognitive levels and shapes the generation prompt.
- A three-role web application spanning student, educator, and administrator surfaces, built with React, TypeScript, and Vite.
- A performance-tracking module that records mastery per Bloom's level and proposes a remedial roadmap.
- An evaluation plan together with a transparent account of what is complete and what remains in progress.

Section II reviews related work. Section III describes the framework. Section IV presents datasets and implementation. Section V describes the evaluation plan and a case study. Section VI discusses limitations, and Section VII concludes.

II. Related Work

A. Retrieval-Augmented Generation

Lewis et al. introduced retrieval-augmented generation, conditioning a generator on passages retrieved from a non-parametric memory to improve factual grounding on knowledge-intensive tasks [3]. Dense retrieval over vector embeddings is now the common way to select such passages, since embeddings capture semantic similarity that keyword overlap misses.

B. LLM-Based Tutoring

LLMs have been applied to question generation, feedback, and conversational tutoring, with products such as Khan Academy's Khanmigo layering a chat tutor on a fixed curriculum. A recurring concern is that ungrounded generation can produce plausible but incorrect or out-of-scope material, motivating grounding in retrieved course content.

C. Bloom's Taxonomy in Assessment

Bloom's taxonomy [1] and its 2001 revision [2] remain the dominant framework for classifying learning objectives by cognitive demand. Prior systems mostly use Bloom's levels as a post-hoc label; fewer use the taxonomy as an explicit generation-time routing signal, which is the role it plays here.

D. Research Gap

Three gaps motivate this design: study aids are rarely grounded in the assigned textbook; cognitive level is treated as a label rather than a control signal; and personalization rarely closes the loop with mastery-tracked remediation. Table I summarizes the response to each gap.

TABLE I RESEARCH GAPS AND SOLUTIONS

Research gap	Aegis Learn design choice
Study aids not grounded in the textbook	RAG over indexed OpenStax/NCERT/Khan corpus
Bloom's level used only after generation	Bloom's level routes the prompt before generation
Personalization limited to difficulty	Per-level mastery tracking + remedial roadmap
No visibility into retrieval	Educator portal exposes corpus/retrieval stats

III. Proposed Framework

A. Overview

Fig. 1 shows the workflow. A user logs in and selects or uploads a textbook corpus; text is chunked and indexed; a query is embedded and compared against the index by cosine similarity to retrieve the top-k chunks; the request is routed to a Bloom's level; a structured prompt is sent to Gemini; and the response drives notes, a quiz, and downstream mastery analysis.

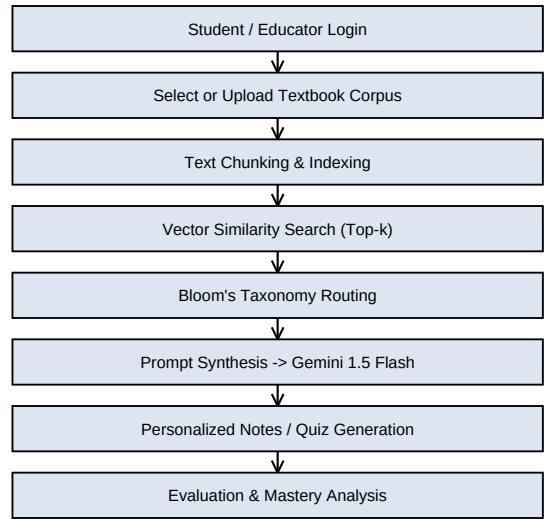


Fig. 1. Aegis Learn system workflow.

B. Bloom's Taxonomy Routing

Fig. 2 shows the six cognitive levels used to route generation [1], [2]. A Remembering-level request asks for definitions and recall items; a Creating-level request asks for open-ended, synthesis tasks built from the same retrieved passages. Routing the level before generation lets one retrieved context serve six different cognitive demands.

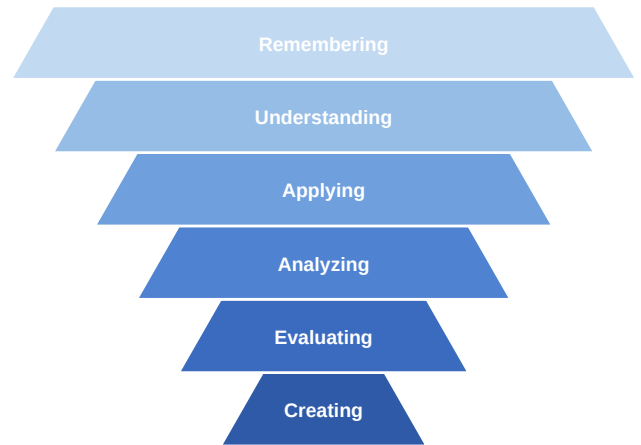


Fig. 2. Bloom's Taxonomy levels used to route prompt generation.

C. Corpus and Retrieval

Reference material is drawn from OpenStax, NCERT, and a Khan Academy algebra corpus (Table II). At query time the request q is embedded into the same space as indexed chunks c_i ; the top-k most similar chunks form the retrieved context R :

$$\text{sim}(q, c_i) = (q \cdot c_i) / (\|q\| \cdot \|c_i\|) \quad (1)$$

$$R = \text{Top-}k(\{\text{sim}(q, c_i)\}, c_i \in I) \quad (2)$$

TABLE II DATASETS

Dataset	Role
OpenStax corpus	General reference text
NCERT corpus	Curriculum-aligned text
Khan Academy algebra	Subject-specific worked material

D. Prompt Synthesis

The prompt builder assembles a template, R, and the Bloom's tag into one request to Gemini 1.5 Flash (Listing 1). A fixed output format keeps notes and quiz items easy to parse and render.

```
SYSTEM: Use only the retrieved context.
BLOOM LEVEL: <one of six levels>
CONTEXT: [1] <passage> ... [k] <passage>
TASK: Generate notes/quiz items for the given
level, grounded only in the context above.
```

Listing 1. Structure of the generation prompt.

E. Application Surfaces

The **student portal** supports topic search, study guides, quizzes, performance tracking, and a remedial roadmap. The **educator portal** supports corpus upload, corpus statistics, and retrieval inspection. The **administrator console** provides usage analytics, API monitoring, and index management.

IV. Datasets, Stack, and Implementation

A. Technology Stack

TABLE III TECHNOLOGY STACK

Layer	Technology
Generation	Gemini 1.5 Flash
Retrieval	RAG, cosine similarity search
Prompting	Bloom's Taxonomy template routing
Front end	React.js, TypeScript, Vite
Styling	Tailwind CSS, Lucide React
Tooling	VS Code, Git, GitHub

The app is a Vite-built React/TypeScript SPA with a conventional layout: `public/` for static assets and `src/` containing `assets`, `components`, `data`, and `utils`, with `App.tsx` and `main.tsx` as entry points. A Gemini API key enables live generation; the UI can still be exercised against local/mock data without one.

B. Development Status

Development follows four phases; the project is currently in Phase 4 (interface development and RAG integration). Fig. 3 summarizes item counts by status. Completed: architecture/objectives, dataset selection, similarity simulator, chunking, Bloom's prompt templates, Gemini integration, and login/navigation. In progress: retrieval analyzer, Bloom-based notes, quiz system, mastery dashboard. Upcoming: educator uploader, validation/testing, documentation, deployment.

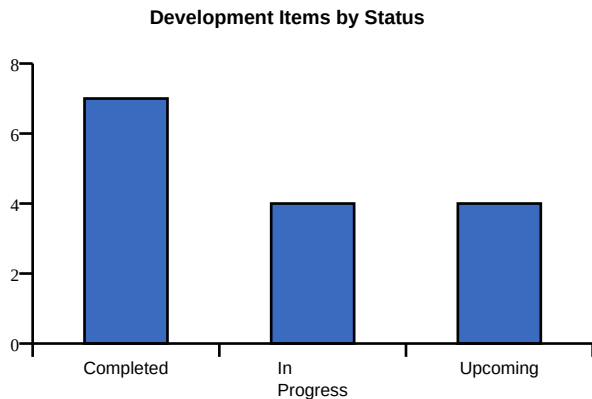


Fig. 3. Development items grouped by current status.

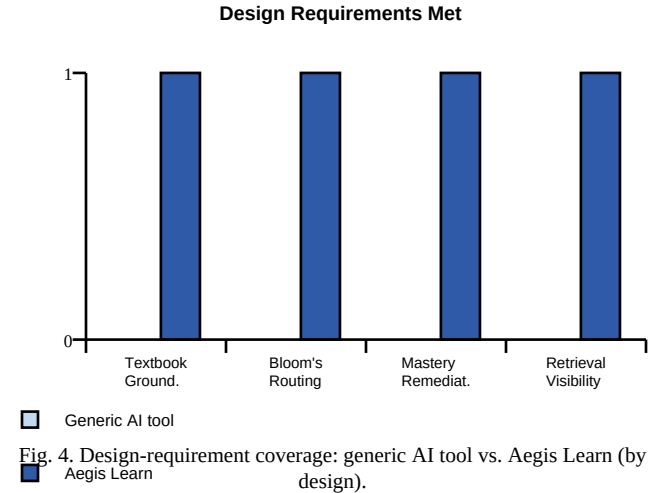
V. Evaluation Plan and Case Study

A. Evaluation Metrics

TABLE IV EVALUATION METRICS

Metric	What it measures
Faithfulness	Content supported by retrieved context
Answer relevance	Content addresses the student's query
Retrieval P/R	Quality of top-k similarity search
Mastery score	Performance across Bloom's levels

Fig. 4 turns Table I into a feature-coverage comparison: a generic, ungrounded AI writing tool meets none of the four design requirements identified above, whereas Aegis Learn is designed to satisfy all four once the in-progress components (Section IV-B) are complete. The chart reflects design intent, not a measured benchmark.



B. Illustrative Walkthrough

A student searching “solving linear equations” at the Applying level triggers retrieval from the indexed Khan Academy corpus, routing through the Applying template, and a rendered study guide plus quiz. Item outcomes are logged per Bloom's level so the mastery module can flag the weakest level and propose remedial notes before the student re-attempts Applying-level questions. This describes intended behavior; the retrieval analyzer, quiz engine, and mastery dashboard that would produce real faithfulness, relevance, and mastery-score numbers are still in progress.

VI. Limitations and Threats to Validity

(i) No quantitative faithfulness, relevance, or retrieval P/R numbers exist yet. (ii) Mastery tracking and remediation are untested with real students. (iii) Corpus coverage beyond OpenStax, NCERT, and Khan algebra is unverified. (iv) Generation depends on the hosted Gemini API, so latency/cost are uncharacterized. (v) No human usefulness study has been run.

VII. Conclusion and Future Work

We presented Aegis Learn, a retrieval-augmented framework that grounds personalized study notes and quizzes in a curated textbook corpus and routes generation through Bloom's Taxonomy. The interface, prompt templates, and API integration are complete; the retrieval analyzer, quiz engine, and mastery dashboard are in progress.

Future work: (1) complete the retrieval analyzer and report faithfulness/relevance/retrieval P/R; (2) finish the quiz engine and mastery dashboard; (3) an educator corpus uploader with validation;

(4) a small human usefulness study; (5) production deployment and cost/latency characterization.

Acknowledgment

The authors thank their guide, Dr. K. Swanthana, Department of Computer Science and Engineering, KL University, Hyderabad, for guidance throughout this project.

References

- [1] B. S. Bloom, Ed., Taxonomy of Educational Objectives, Handbook I: Cognitive Domain. New York, NY, USA: David McKay Company, 1956.
- [2] L. W. Anderson and D. R. Krathwohl, Eds., A Taxonomy for Learning, Teaching, and Assessing. New York, NY, USA: Longman, 2001.
- [3] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *NeurIPS*, vol. 33, 2020, pp. 9459-9474.
- [4] Gemini Team, Google, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *arXiv:2403.05530*, 2024.
- [5] "OpenStax: Free textbooks, free." [Online]. Available: <https://openstax.org>
- [6] "NCERT." [Online]. Available: <https://ncert.nic.in>
- [7] "Khan Academy." [Online]. Available: <https://www.khanacademy.org>
- [8] "React." [Online]. Available: <https://react.dev>
- [9] "Vite." [Online]. Available: <https://vite.dev>
- [10] "Tailwind CSS." [Online]. Available: <https://tailwindcss.com>